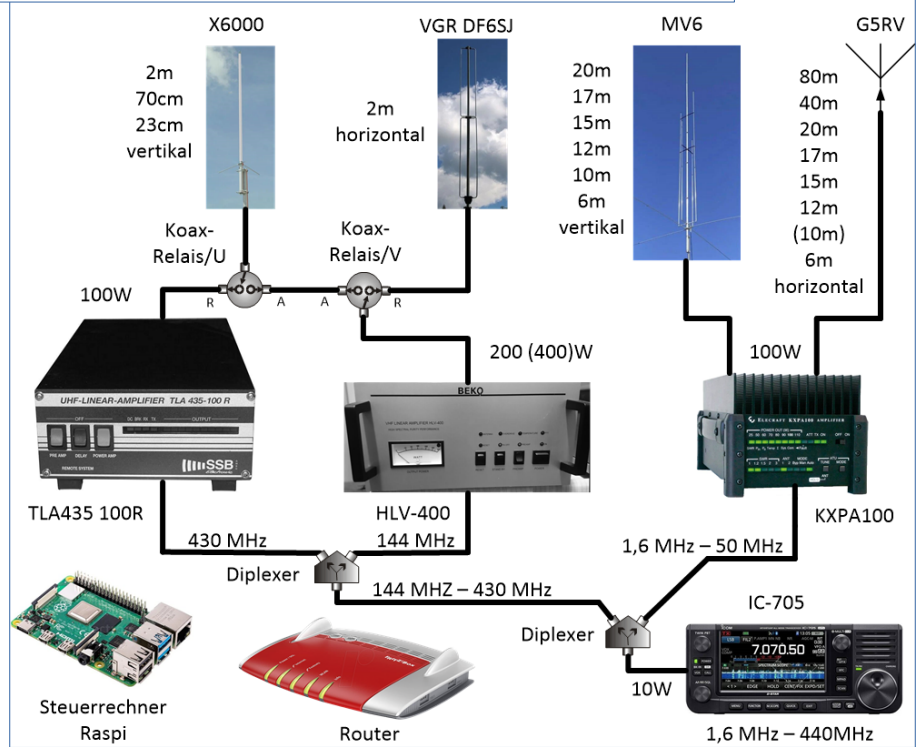


CAT – Steuerung an Beispielen



- ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- YAESU FT-710 mit externem Tuner
 - Steuerung über PC
 - Steuerung über ESP32
- Verweise

Remote- Station mit IC-705



02.08.26 dk2jk

2/22

ajsjjsjjs
dlhdlflkf
fljdfjdjsfj
sdfjölkjöjölj

Aufgabe der Steuerung

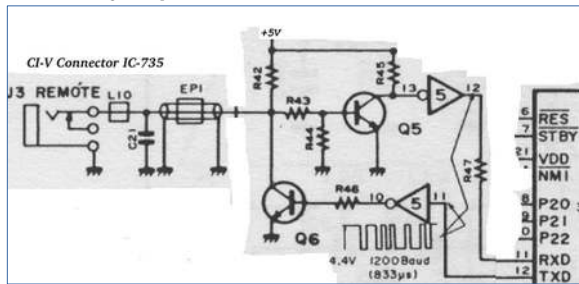
- Frequenzabfrage via CAT- Control

Daraus ableiten:

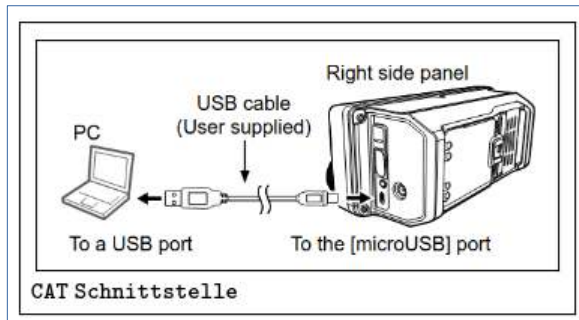
- PTT-Freigabe
- Tuner Voreinstellung KW
- Antennen-Relais
- Leistungssteuerung

Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



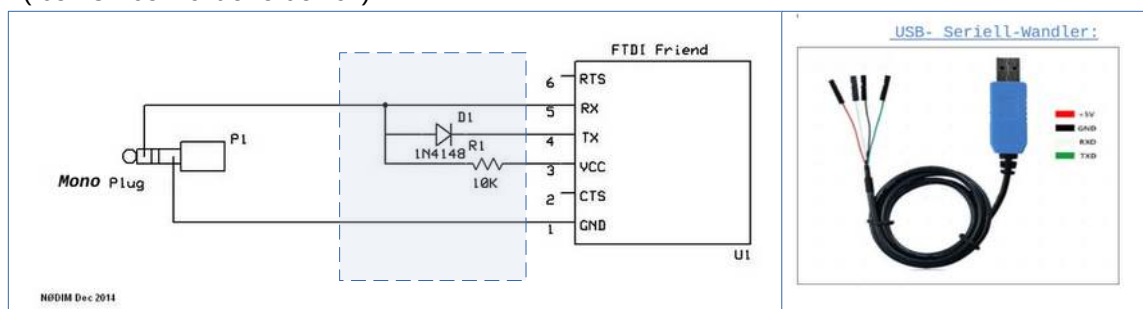
CI-V „Computer Interface V (Fünf) TTL -Level (Ruhezustand ca +5V, Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung (RXD und TXD) wird auf eine Leitung kombiniert (Bus).



Beim IC-705 ist stattdessen eine virtuelle serielle Schnittstelle als **USB-Schnittstelle** vorhanden

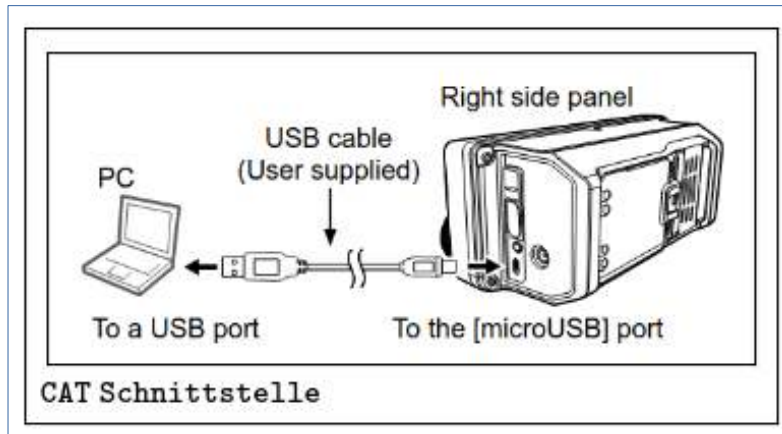
CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)



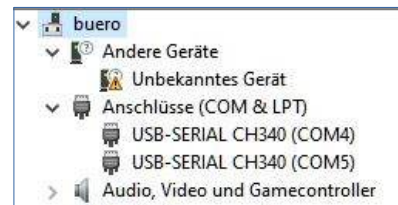
Adapter	---	CI-V Adapter
GND ,schwarz	---	G
TXD ,grün	---	rx
RXD ,weiß	---	tx
+5V	---	:

USB als virtueller COM-Port

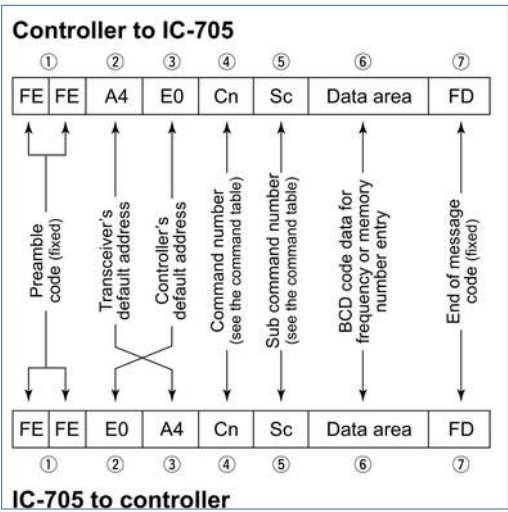


```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```

02.08.26 dk2jk



CI-V Command Table ICOM-705



◇ Command table

03*1	See p. 16.	Read the operating frequency
------	------------	------------------------------

Page 16:

◇ Command formats

• Operating frequency

Command: 00, 03, 05, 1C 03

①	②	③	④	⑤
X	X	X	X	X
10 Hz digit: 0 - 9	1 Hz digit: 0 - 9	1 kHz digit: 0 - 9	100 Hz digit: 0 - 9	100 kHz digit: 0 - 9
10 kHz digit: 0 - 9	10 MHz digit: 0 - 9	1 MHz digit: 0 - 9	1 GHz digit: (fixed)	100 MHz digit: 0 - 4

Frequenzabfrage Datenformat

```
#ICOM Kommando zum Frequenz lesen:  
# FE FE A4 E0 03 FD  
# FE FE          <- kopf  
# ..... A4      <- IC-705 CAT Adresse  
# ..... E0      <- default ??  
# ..... 03      <- Kommando Frequenz lesen  
# ..... FD      <- Ende Zeichen
```


Frequenzabfrage ICOM Python Programm

```
[ ic705_cat_mit kommentar.py ]
1
2 from serial import Serial      # modul serial laden
3
4
5 def read_frequenz_IC_705():    # Kommentar mit Beispieldaten...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD] # Frequenz lesen ...
7                                     # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )       # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]           #
12                                     # 00 50 12 07 00
13    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 07 12 50 00
14    text = nutzdaten_rueckwaerts.hex()  # text = "0007125000"
15    mhz = float(text)/1000_000          # mhz = float("0007125000")/1e6
16    return mhz                        # schnittstelle schliessen
17                                     # ergebnis = 7.125
18
19 if __name__ == '__main__':
20     frequenz = read_frequenz_IC_705() # frequenz lesen
21     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz

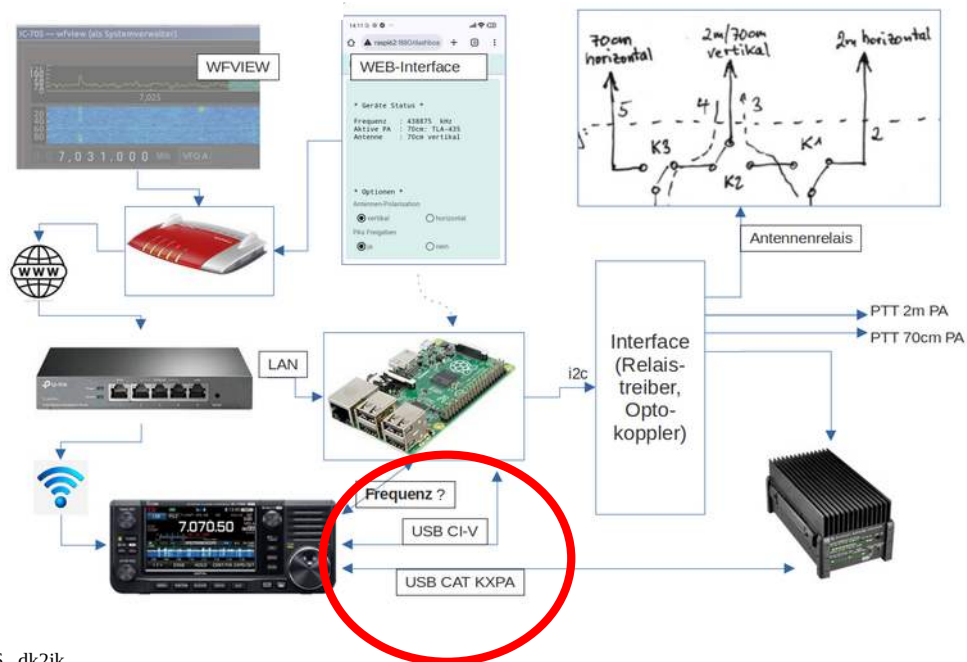
Shell
>>> %Run 'ic705_cat_mit kommentar.py'
f=7.125 MHz
```

02.08.26 Onzejn

Frequenzabfrage kommentiert

```
CAT-Kommando = FE FE A4 E0 03 FD  <= Frequenz lesen
Antwort       = FE FE E0 A4 03 00 50 12 07 00 FD
Nutzdaten     = 00 50 12 07 00
Nutzdaten     = 00 07 12 50 00 (gedreht)
Roh-Ergebnis = "0007125000" = text
Frequenz      = 7.125 MHz      = float(text)/1000000
```

Wie hängt alles zusammen?



CAT-Control KXPA100

Befehlsformat

KXPA100-Befehle beginnen in der Regel mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon abgeschlossen (;).

Hier wird verwendet:

^MT (ATU-Speicherabruf)
^MTffff; wobei fffff eine Frequenz in kHz ist.

**Der Tuner wird auf die Frequenz geschaltet
noch bevor er überhaupt HF sieht.**

über CAT-Schnittstelle
(einfaches ASCII-Format)

Befehl: ^MT7032;
== Memory Tune 7032 kHz

Abstimmen eines ext. Tuners am FT-710

(Tuner ist ein SG-230)

Möglichkeiten:

1. Manuell:

Mode=AM (ca. 25W) --> PTT am Mikrofon drücken --
>Tuner Arbeitet --> zurück auf alten ,Mode'

2. Verwendung der Buchse „REM/ALC“

Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am
Signal ALC; -4.9V --> 10W. (TUN/LIN PORT SELECT
auf ,Linear')

3. CAT-Schnittstelle:

— eine Kommandofolge senden, die den manuellen
Tune-Vorgang nachbildet.

Frequenzabfrage YAESU Python Programm

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

```
cat_pc_basic.py <
12
13 port = '/dev/ttyUSB0'      # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
20     time.sleep(0.2)           # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()   # bytes in ascii umwandeln
23
```

```
Shell <
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

02.08.26 dk2jk

Experimente live

```
Shell >
>>> cat()          Frequenz ?
                    (default Argument)
'fa007029980;'

>>> cat('md0;')    Mode?
'md07;'

>>> cat('pc;')      Power?
'pc077;'

>>> cat('pc099;')   Power =99W
''

>>> cat('pc;')      Power ?
'pc099;'

>>> cat('fa007039000;') Frequenz = ...!
''

>>> cat()          Frequenz ?
'fa007039000;'
```

```
def cat( cmd = 'fa;'):
    my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
    time.sleep(0.2)           # etwas warten
    antwort = my_ser.read_all() # und antwort lesen
    return antwort.decode()    # bytes in ascii umwandeln
```

02.08.26 dk2jk

Tune Programm für YAESU FT-710 am PC

Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

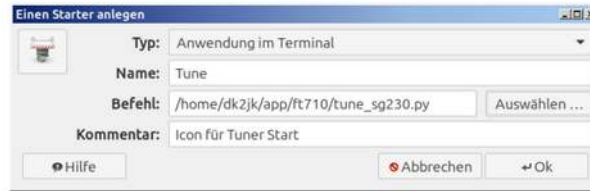
```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write ( beep )  
    close() # Schnittstelle schliessen  
    return
```

Komplettes Programm im
Anhang

Tune Icon Link (Linux)

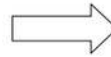
Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

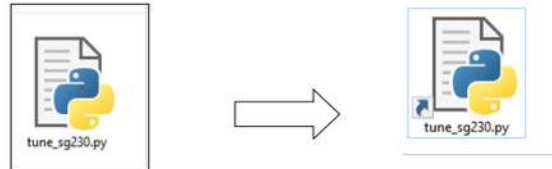
```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon = python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:



Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

4-pol. Kabel (nur für Dokumentation)

FT-710
CAT-3 Connector
8-Pol-Mini-Din¹
Cat-3.1

Cat-3.2
Cat-3.3
Cat-3.4

Cat-3.5
Cat-3.6
Cat-3.7
Cat-3.8

TUNER/LINEAR Jack

5 Volt vom USB des
ESP-32- Moduls

5V_USB

Convert to PCB : no

U2 LM7805T

13_VB

VIN

GND

OUT

3

D2

1N4001

5V_ESP

C1 22uF

C2 100nF

D3 1N4001

R1 2k2

R2 2k2

R3 2k2

R4 220

R5 2k2

LED

3_3V

U1

GPIO5 5V
GPIO6 GND
GPIO7 3V3
GPIO8
GPIO4 3.3 Volt Regler
GPIO3 befindet
GPIO2 sich auf dem
GPIO1 ESP-32 Modul
GPIO0

ESP32-C3

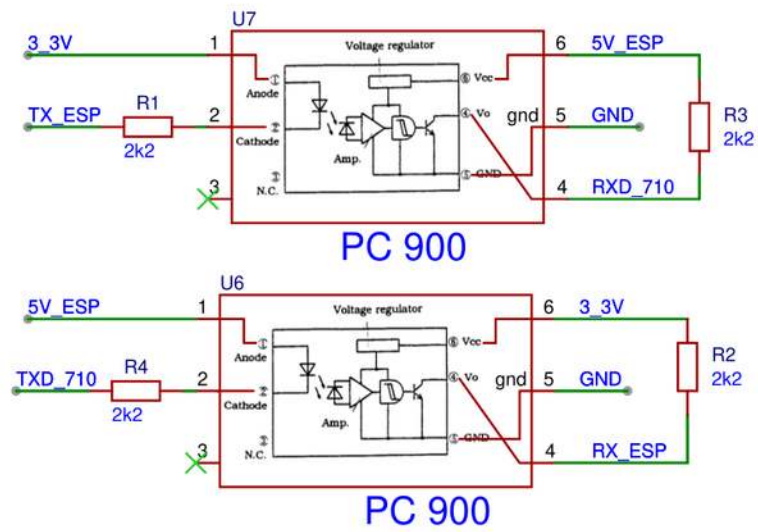
H2 Taster
H3

Hole3mm.1
Hole3mm.2
Hole3mm.3

(as viewed from rear panel)

02.08.26 dk2jk

Variante mit Optokoppler



MicroPython Programm ESP32

```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write ( beep )
    close()
    return
```

```
def main():
    x= taste() # taste lesen , low aktiv
    led(x) # led an solange taste gedrueckt ist
    if x ==0: # taste gedrueckt ?
        tune_function()
```

Verweise

tbd.